



TRABALHO PRÁTICO DE INTRODUÇÃO ÀS REDES DE COMPUTADORES

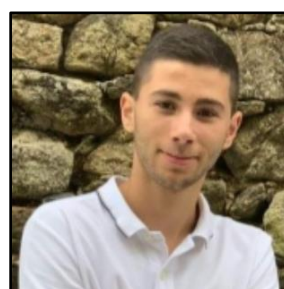
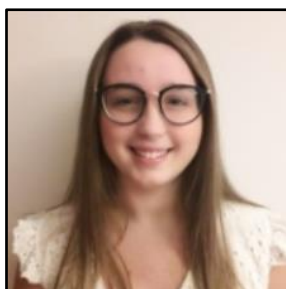
SIMULAÇÃO DE LANS ETHERNET E REDES TCP/IP USANDO O CORE

Ano letivo: 2024/2025 | 2º Semestre

Unidade curricular: Introdução às Redes de Computadores

Turno prático / Grupo: PL1 / Grupo 04

Realizado por: Catarina Carvalho de Sousa - a97259,
Eric Masaro Nakamura Bafa - A108780,
Ezequiel Alexandre Pereira Martins - A97125





Introdução

O presente trabalho prático foi realizado no âmbito da unidade curricular de Introdução às Redes de Computadores, tendo como principal objetivo aplicar, de forma prática, os conceitos teóricos abordados ao longo do semestre. Para a concretização deste projeto, recorreu-se à utilização de uma máquina virtual, na qual se utilizou o emulador CORE, ferramenta que permitiu a simulação de LANs Ethernet e redes TCP/IP.

1. Emulação de LANs Ethernet

- a) Nesta primeira fase, emulou-se uma rede local, através da utilização do CORE, de forma a perceber o funcionamento dos Hubs e Switches. Deste modo, foi permitido construir-se (em modo de edição) a topologia de rede pretendida, demonstrada na Figura 1, constituída por dois Hubs (H1 e H2) e dois Switches (S1 e S2). Os dispositivos identificados pelas letras A, B, C e D são PCs.

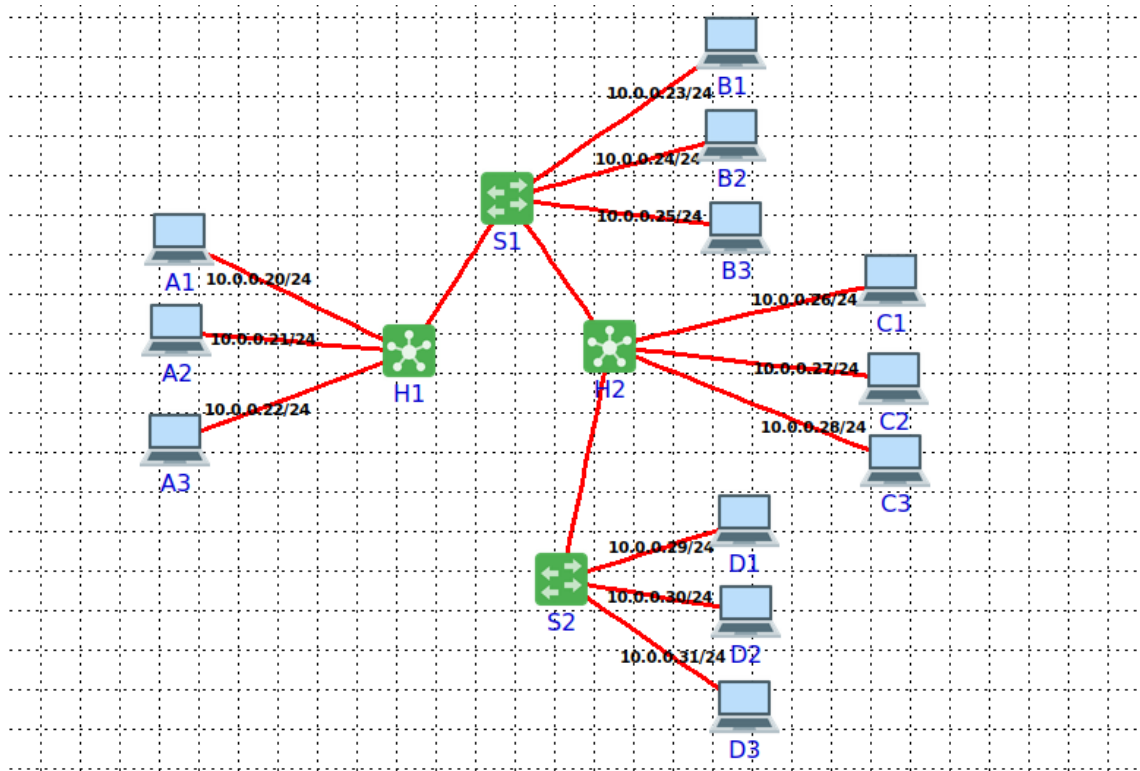


Figura 1- Topologia de rede local usando dois Hubs (H1 e H2) e dois Switches (S1 e S2)



- b) Aceitaram-se os endereços IPv4 que foram atribuídos automaticamente a cada uma das interfaces de rede e para cada um dos PCs, foram desativados os endereços IPv6, tal como demonstrado na Figura 2, para o PC A1.

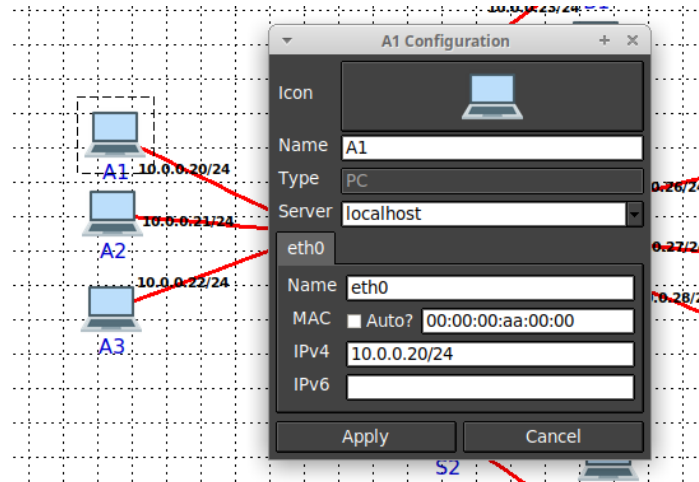


Figura 2 - Desativação dos endereços IPv6

- c) Para verificar o funcionamento da topologia implementada, foram realizados testes de conectividade entre diferentes terminais através do comando *ping*, de forma a verificar o correto funcionamento da rede. Na Figura 3, apresenta-se um dos testes realizados, em que o terminal A1 (10.0.0.20) envia três pacotes para o terminal B2 (10.0.0.24). Como se pode observar, todos os pacotes foram entregues com sucesso, sem perda de pacotes, confirmando a ligação funcional entre os dois nós da rede.

```

Terminal - root@A1: /tmp/pycore.1/A1.conf
File Edit View Terminal Tabs Help
root@A1:/tmp/pycore.1/A1.conf# ping -w3 10.0.0.24
PING 10.0.0.24 (10.0.0.24) 56(84) bytes of data:
64 bytes from 10.0.0.24: icmp_seq=1 ttl=64 time=0.153 ms
64 bytes from 10.0.0.24: icmp_seq=2 ttl=64 time=0.101 ms
64 bytes from 10.0.0.24: icmp_seq=3 ttl=64 time=0.052 ms

--- 10.0.0.24 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2081ms
rtt min/avg/max/mdev = 0.052/0.102/0.153/0.041 ms
root@A1:/tmp/pycore.1/A1.conf#
    
```

Figura 3 - Teste de conectividade (A1 - B2)



Enquadramento teórico

O protocolo ARP (Address Resolution Protocol) permite que um dispositivo descubra o endereço físico (MAC) correspondente a um endereço lógico (IP). Quando um PC quer comunicar com outro dentro da mesma rede, precisa de conhecer o endereço MAC do destino. Se souber apenas o IP, envia um pedido ARP do tipo "Who has <10.0.0.24> ? Tell <10.0.0.20 >".

De modo a ilustrar o funcionamento do protocolo ARP foi efetuada e analisada uma captura de tráfego (Wireshark), onde foi possível observar o funcionamento do ARP entre dois computadores da rede, A1 e B2.

Foi aplicado um filtro "arp", de modo a conseguir analisar-se apenas tráfego correspondente a esse protocolo.

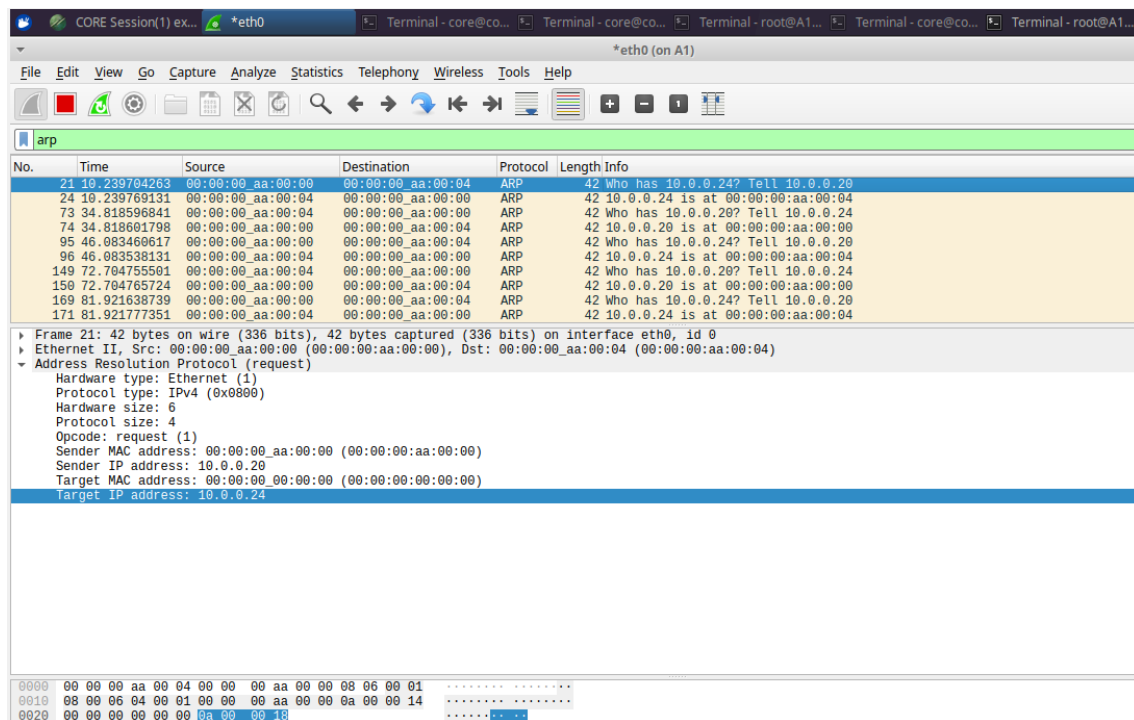


Figura 4 - Captura de tráfego

Na Figura 4, pode verificar-se que o PC A1 com IP 10.0.0.20 quer enviar dados para o IP 10.0.0.24 (B2), mas não conhece o seu MAC. Para o descobrir, envia uma mensagem ARP em broadcast: *Who has 10.0.0.24? Tell 10.0.0.20*. O dispositivo com o IP 10.0.0.24 responde: *10.0.0.24 is at 00:00:00:aa:00:04*. Assim, o computador 10.0.0.20 consegue associar o IP 10.0.0.24 ao MAC 00:00:00:aa:00:04. De seguida, também se verifica o processo inverso: o PC 10.0.0.24 envia um pedido ARP para descobrir o MAC de 10.0.0.20, obtendo como resposta: *10.0.0.20 is at 00:00:00:aa:00:00*.



2. Interligação de LANs e redes IP

a) Neste exercício é proposto criar uma topologia de rede com cinco routers (RA, RB, RC, RD e RE) aos quais devem estar ligadas LANs e quatro routers que servem como equipamentos de interligação (R1, R2, R3 e R4).

b)

Rede	Nº de endereços
A	$2^{32-(17+1)} - 3 = 16381$
B	$2^{32-(17+2)} + 2^{32-(17+4)} - 6 = 10234$
C	$2^{32-(17+3)} - 3 = 4093$
D	$2^{32-(17+5)} - 3 = 1021$
E	$2^{32-(17+5)} - 3 = 1021$

Atribuição de Endereços IP

Para a atribuição de endereços IP às redes A, B, C, D e E foi utilizada a gama 10.4.0.0/17, conforme indicado no enunciado, tendo em conta que G=04 e T=1, resultando numa máscara base de /17, ($M = 16 + T$).

A rede A requer o maior número de endereços. A rede B foi dividida em duas sub-redes (B1 e B2).

Para cada rede foi selecionada a máscara que melhor se ajusta ao número de endereços necessários:

- A → /18 ($32 - 18 = 14$ // $2^{14} = 16384$)
- B1 → /19 ($32 - 19 = 13$ // $2^{13} = 8192$)
- B2 → /21 ($32 - 21 = 11$ // $2^{11} = 2048$)
- C → /20 ($32 - 20 = 12$ // $2^{12} = 4096$)
- D → /22 ($32 - 22 = 10$ // $2^{10} = 1024$)
- E → /22 ($32 - 22 = 10$ // $2^{10} = 1024$)

Para cada rede foram definidos: O endereço da rede, o endereço de difusão, um IP para o router (o primeiro IP utilizável) e a gama de endereços utilizáveis entre o IP da rede e o IP difusão.



A Tabela 1 resume a distribuição realizada:

Rede	IP Rede	Máscara	IP Difusão	IP Router	Gama
A	10.4.0.0	/18	10.4.63.255	10.4.0.1	10.4.0.1 <> 10.4.63.254
B1	10.4.64.0	/19	10.4.95.255	10.4.64.1	10.4.64.1 <> 10.4.95.254
B2	10.4.96.0	/21	10.4.103.255	10.4.96.1	10.4.96.1 <> 10.4.103.254
C	10.4.104.0	/20	10.4.119.255	10.4.104.1	10.4.104.1 <> 10.4.119.254
D	10.4.120.0	/22	10.4.123.255	10.4.120.1	10.4.120.1 <> 10.4.123.254
E	10.4.124.0	/22	10.4.127.255	10.4.124.1	10.4.124.1 <> 10.4.127.254

Para as ligações entre os *routers*, foi utilizada a gama de endereços 192.168.0.0/24, conforme especificado no enunciado. Como estas ligações requerem apenas dois endereços IP por rede, foi utilizada a máscara /30, que permite dois *hosts* por sub-rede.

A Tabela 1 mostra os pares de routers interligados e os respectivos endereços IP atribuídos:

Ligação	IP	Ligação	IP
RA - R1	192.168.0.1/30	R4 - R2	192.168.0.26/30
R1 - RA	192.168.0.2/30	R3 - RC	192.168.0.29/30
R1 - RC	192.168.0.5/30	RC - R3	192.168.0.30/30
RC - R1	192.168.0.6/30	R3 - RE	192.168.0.33/30
R1 - R3	192.168.0.9/30	RE - R3	192.168.0.34/30
R3 - R1	192.168.0.10/30	R3 - R4	192.168.0.37/30
R1 - R2	192.168.0.13/30	R4 - R3	192.168.0.38/30
R2 - R1	192.168.0.14/30	R4 - RD	192.168.0.41/30
R2 - RB	192.168.0.17/30	RD - R4	192.168.0.42/30
RB - R2	192.168.0.18/30	RD - RE	192.168.0.45/30
R2 - R3	192.168.0.21/30	RE - RD	192.168.0.46/30
R3 - R2	192.168.0.22/30	RC - RE	192.168.0.49/30
R2 - R4	192.168.0.25/30	RE - RC	192.168.0.50/30

Tabela 1-Endereços das redes A, B1, B2, C, D, E



- c) Com base na topologia definida e no plano de endereçamento estabelecido, foi elaborado um esquema de encaminhamento para cada router da rede. Cada tabela de encaminhamento apresenta, para cada rede de destino, a máscara correspondente, a interface de saída e o próximo nó (IP do router para onde os pacotes devem ser enviados).

Para o **router A**, foi elaborada a Tabela 2:

	Rede destino	Máscara	Interface de Saída	Próximo nó
A	10.4.0.0	/18	10.4.0.1	----
B1	10.4.64.0	/19	192.168.0.1	192.168.0.2
B2	10.4.96.0	/21	192.168.0.1	192.168.0.2
C	10.4.104.0	/20	192.168.0.1	192.168.0.2
D	10.4.120.0	/22	192.168.0.1	192.168.0.2
E	10.4.124.0	/22	192.168.0.1	192.168.0.2

Tabela 2 - Router A

Para o **router B**, foi elaborada a Tabela 3:

	Rede destino	Máscara	Interface de Saída	Próximo nó
A	10.4.0.0	/18	192.168.0.18	192.168.0.17
B1	10.4.64.0	/19	10.4.64.1	----
B2	10.4.96.0	/21	10.4.96.1	----
C	10.4.104.0	/20	192.168.0.18	192.168.0.17
D	10.4.120.0	/22	192.168.0.18	192.168.0.17
E	10.4.124.0	/22	192.168.0.18	192.168.0.17

Tabela 3- Router B

Para o **router C**, foi elaborada a Tabela 4:

	Rede destino	Máscara	Interface de Saída	Próximo nó
A	10.4.0.0	/18	192.168.0.6	192.168.0.5
B1	10.4.64.0	/19	192.168.0.6	192.168.0.5
B2	10.4.96.0	/21	192.168.0.6	192.168.0.5
C	10.4.104.0	/20	10.4.104.1	----
D	10.4.120.0	/22	192.168.0.49	192.168.0.50
E	10.4.124.0	/22	192.168.0.49	192.168.0.50

Tabela 4- Router C



Para o **router D**, foi elaborada a Tabela 5:

	Rede destino	Máscara	Interface de Saída	Próximo nó
A	10.4.0.0	/18	192.168.0.42	192.168.0.41
B1	10.4.64.0	/19	192.168.0.42	192.168.0.41
B2	10.4.96.0	/21	192.168.0.42	192.168.0.41
C	10.4.104.0	/20	192.168.0.45	192.168.0.46
D	10.4.120.0	/22	10.4.120.1	----
E	10.4.124.0	/22	192.168.0.45	192.168.0.46

Tabela 5- Router D

Para o **router E**, foi elaborada a Tabela 6:

	Rede destino	Máscara	Interface de Saída	Próximo nó
A	10.4.0.0	/18	192.168.0.34	192.168.0.33
B1	10.4.64.0	/19	192.168.0.34	192.168.0.33
B2	10.4.96.0	/21	192.168.0.34	192.168.0.33
C	10.4.104.0	/20	192.168.0.50	192.168.0.49
D	10.4.120.0	/22	192.168.0.46	192.168.0.45
E	10.4.124.0	/22	10.4.124.1	----

Tabela 6 - Router E

Para o **router 1**, foi elaborada a Tabela 7:

	Rede destino	Máscara	Interface de Saída	Próximo nó
A	10.4.0.0	/18	192.168.0.2	192.168.0.1
B1	10.4.64.0	/19	192.168.0.13	192.168.0.14
B2	10.4.96.0	/21	192.168.0.13	192.168.0.14
C	10.4.104.0	/20	192.168.0.5	192.168.0.6
D	10.4.120.0	/22	192.168.0.9	192.168.0.10
E	10.4.124.0	/22	192.168.0.9	192.168.0.10

Tabela 7 - Router 1

Para o **router 2**, foi elaborada a Tabela 8:

	Rede destino	Máscara	Interface de Saída	Próximo nó
A	10.4.0.0	/18	192.168.0.14	192.168.0.13
B1	10.4.64.0	/19	192.168.0.17	192.168.0.18
B2	10.4.96.0	/21	192.168.0.17	192.168.0.18
C	10.4.104.0	/20	192.168.0.14	192.168.0.13
D	10.4.120.0	/22	192.168.0.25	192.168.0.26
E	10.4.124.0	/22	192.168.0.21	192.168.0.22

Tabela 8 - Router 2



Para o **router 3**, foi elaborada a Tabela 9:

	Rede destino	Máscara	Interface de Saída	Próximo nó
A	10.4.0.0	/18	192.168.0.10	192.168.0.09
B1	10.4.64.0	/19	192.168.0.22	192.168.0.21
B2	10.4.96.0	/21	192.168.0.22	192.168.0.21
C	10.4.104.0	/20	192.168.0.29	192.168.0.30
D	10.4.120.0	/22	192.168.0.37	192.168.0.38
E	10.4.124.0	/22	192.168.0.33	192.168.0.34

Tabela 9 - Router 3

Para o **router 4**, foi elaborada a Tabela 10:

	Rede destino	Máscara	Interface de Saída	Próximo nó
A	10.4.0.0	/18	192.168.0.38	192.168.0.37
B1	10.4.64.0	/19	192.168.0.26	192.168.0.25
B2	10.4.96.0	/21	192.168.0.26	192.168.0.25
C	10.4.104.0	/20	192.168.0.38	192.168.0.37
D	10.4.120.0	/22	192.168.0.41	192.168.0.42
E	10.4.124.0	/22	192.168.0.41	192.168.0.42

Tabela 10 - Router 4

Política de Encaminhamento

A política de encaminhamento adotada para a construção das tabelas de encaminhamento foi baseada no menor número de saltos. Sempre que existia mais do que um caminho possível entre o *router* e a rede, foi selecionado aquele que envolvia menos routers intermédios até atingir a rede.

Esta política foi escolhida por ser simples e eficaz, garantindo:

- Redução de tempos de atraso, ao minimizar o número de saltos.
- Encaminhamento eficiente e direto, sem caminhos redundantes.
- Facilidade de implementação.



- d) Neste exercício foi pedido que se desativasse o encaminhamento dinâmico (serviços OSPFv2 e OSPFv3) em todos os *routers* e se adicionasse manualmente as rotas (encaminhamento estático).

A topologia desenvolvida está representada na **Erro! A origem da referência não foi encontrada..**

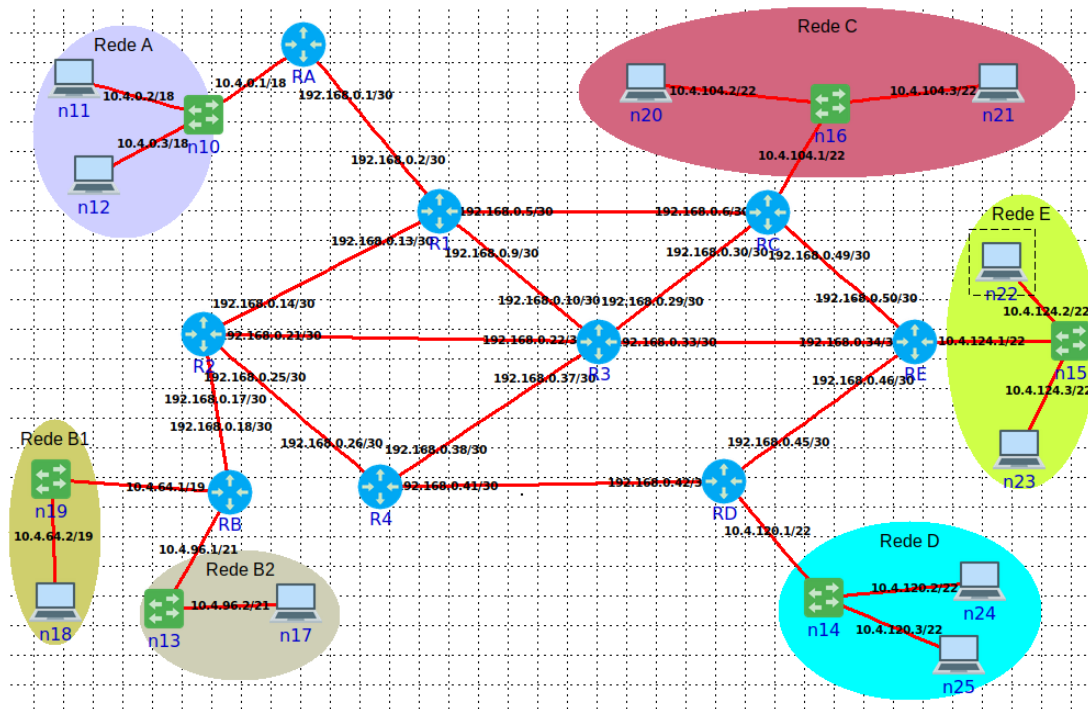
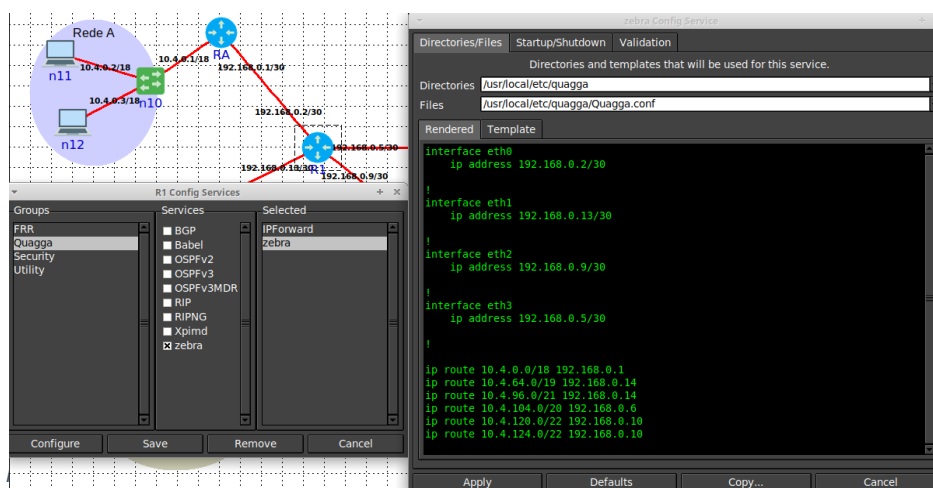


Figura 5-Topologia de rede 2

Na **Erro! A origem da referência não foi encontrada.**, pode ver-se qual foi o procedimento para o *router* 1. Foi utilizado o mesmo método para todos os *routers* restantes. Após selecionar o *router*, foi acedida a opção *Config Services, Quagga, Zebra*, onde se adicionaram as rotas e se desativou o encaminhamento dinâmico.





- e) Efetuaram-se testes de conectividade, entre todas as redes, com os comandos *ping* e *traceroute*.

Na Figura 7 estão representados os testes efetuados no PC n11, da Rede A.

```

Terminal - root@n11: /tmp/pycore.1/n11.conf
File Edit View Terminal Tabs Help

root@n11:/tmp/pycore.1/n11.conf# traceroute 10.4.64.2
traceroute to 10.4.64.2 (10.4.64.2), 30 hops max, 60 byte packets
 1 10.4.0.1 (10.4.0.1) 0.493 ms 0.436 ms 0.426 ms
 2 192.168.0.2 (192.168.0.2) 0.417 ms 0.387 ms 0.375 ms
 3 192.168.0.14 (192.168.0.14) 0.364 ms 0.334 ms 0.321 ms
 4 192.168.0.18 (192.168.0.18) 0.320 ms 0.260 ms 0.246 ms
 5 10.4.64.2 (10.4.64.2) 0.231 ms 0.193 ms 0.176 ms
root@n11:/tmp/pycore.1/n11.conf# traceroute 10.4.96.2
traceroute to 10.4.96.2 (10.4.96.2), 30 hops max, 60 byte packets
 1 10.4.0.1 (10.4.0.1) 0.341 ms 0.296 ms 0.285 ms
 2 192.168.0.2 (192.168.0.2) 0.275 ms 0.256 ms 0.245 ms
 3 192.168.0.14 (192.168.0.14) 0.235 ms 0.215 ms 0.203 ms
 4 192.168.0.18 (192.168.0.18) 0.191 ms 0.169 ms 0.155 ms
 5 10.4.96.2 (10.4.96.2) 0.140 ms 0.090 ms 0.073 ms
root@n11:/tmp/pycore.1/n11.conf# traceroute 10.4.104.2
traceroute to 10.4.104.2 (10.4.104.2), 30 hops max, 60 byte packets
 1 10.4.0.1 (10.4.0.1) 0.050 ms 0.009 ms 0.005 ms
 2 192.168.0.2 (192.168.0.2) 0.013 ms 0.006 ms 0.006 ms
 3 192.168.0.6 (192.168.0.6) 0.029 ms 0.009 ms 0.008 ms
 4 10.4.104.2 (10.4.104.2) 0.035 ms 0.011 ms 0.012 ms
root@n11:/tmp/pycore.1/n11.conf# traceroute 10.4.120.3
traceroute to 10.4.120.3 (10.4.120.3), 30 hops max, 60 byte packets
 1 10.4.0.1 (10.4.0.1) 0.628 ms 0.586 ms 0.574 ms
 2 192.168.0.2 (192.168.0.2) 0.563 ms 0.543 ms 0.532 ms
 3 192.168.0.10 (192.168.0.10) 0.520 ms 0.455 ms 0.438 ms
 4 192.168.0.38 (192.168.0.38) 0.424 ms 0.356 ms 0.268 ms
 5 192.168.0.42 (192.168.0.42) 0.228 ms 0.189 ms 0.171 ms
 6 10.4.120.3 (10.4.120.3) 0.134 ms 0.023 ms 0.016 ms
root@n11:/tmp/pycore.1/n11.conf# traceroute 10.4.124.2
traceroute to 10.4.124.2 (10.4.124.2), 30 hops max, 60 byte packets
 1 10.4.0.1 (10.4.0.1) 0.404 ms 0.322 ms 0.313 ms
 2 192.168.0.2 (192.168.0.2) 0.302 ms 0.282 ms 0.270 ms
 3 192.168.0.10 (192.168.0.10) 0.260 ms 0.236 ms 0.221 ms
 4 192.168.0.34 (192.168.0.34) 0.209 ms 0.168 ms 0.153 ms
 5 10.4.124.2 (10.4.124.2) 0.138 ms 0.092 ms 0.074 ms
root@n11:/tmp/pycore.1/n11.conf# ping -w5 10.4.120.3
PING 10.4.120.3 (10.4.120.3) 56(84) bytes of data:
64 bytes from 10.4.120.3: icmp_seq=1 ttl=59 time=0.147 ms
64 bytes from 10.4.120.3: icmp_seq=2 ttl=59 time=0.070 ms
64 bytes from 10.4.120.3: icmp_seq=3 ttl=59 time=0.102 ms
64 bytes from 10.4.120.3: icmp_seq=4 ttl=59 time=0.075 ms
64 bytes from 10.4.120.3: icmp_seq=5 ttl=59 time=0.076 ms

--- 10.4.120.3 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4098ms
rtt min/avg/max/mdev = 0.070/0.094/0.147/0.028 ms
root@n11:/tmp/pycore.1/n11.conf#

```

Figura 7 - Testes de conectividade PC rede A

Foi utilizado o comando *traceroute* desde um pc da rede A para um pc de todas as outras redes. Foi possível verificar que o caminho percorrido estaria de acordo com as tabelas de encaminhamento efetuadas. Foi também repetido o mesmo processo entre todos os pc's das outras redes. De seguida, foi utilizado o comando *ping* para um pc da rede D e verificou-se que a transmissão de pacotes foi bem sucedida.



- f) Considerando que a LAN E, 10.4.124.0/22, atingiu o número máximo de dispositivos, configurou-se a rede para permitir a adição de mais dispositivos. Para isso, foi utilizada a rede 192.168.100.0/26. Dois novos PCs foram ligados ao mesmo *switch* já utilizado na LAN E, sendo atribuídos os endereços 192.168.100.2/26 e 192.168.100.3/26 e *gateway* configurado como 192.168.100.1, como pode verificar-se na Figura 8. Para garantir a comunicação entre os novos dispositivos e o resto da rede, foi configurado no router RE, que já estava ligado à LAN E, um segundo endereço IP na mesma interface física, correspondente ao *gateway* da nova sub-rede (192.168.100.1/26). Assim, o router RE passou a ter simultaneamente um endereço da rede original (10.4.124.1/22) e da nova rede (192.168.100.1/26) na mesma interface. Estas configurações podem ver-se na **Erro! A origem da referência não foi encontrada..**

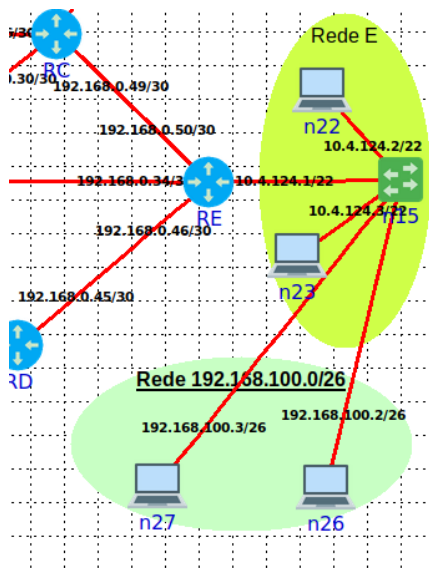


Figura 8 - Rede 192.168.100.0/26

```
interface eth3
  ip address 10.4.124.1/22
  ip address 192.168.100.1/26
!
```

Figura 11 - Configurações Router E

Como os dispositivos estão ligados ao mesmo switch mas pertencem a sub-redes IP diferentes, quando um dispositivo da rede 192.168.100.0/26 pretende comunicar com outro da rede 10.4.124.0/22, o pacote é enviado ao seu *gateway*, que neste caso é o router RE. Este, por ter endereços configurados nas duas redes, consegue assegurar a comunicação, como pode ver-se na Figura 9, abaixo

```
root@n26:/tmp/pycore.1/n26.conf# traceroute 10.4.124.2
traceroute to 10.4.124.2 (10.4.124.2), 30 hops max, 60 byte packets
 1 192.168.100.1 (192.168.100.1) 0.246 ms 0.206 ms 0.196 ms
 2 10.4.124.2 (10.4.124.2) 0.185 ms 0.164 ms 0.150 ms
```

Figura 9 - Traceroute n26-n22



3. DHCP – Dynamic Host Configuration Protocol

- a) De forma a realizar o objetivo deste exercício que seria ativar a configuração automática dos endereços IP recorrendo ao protocolo DHCP, alterou-se a topologia da Rede B1, para que tivesse três PCs, como mostra a Figura 10.

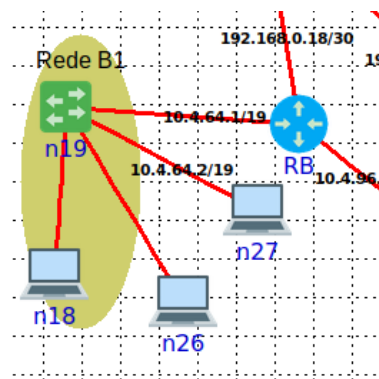


Figura 10 - Topologia rede B1

- b) Foi configurado o PC n27 (IP 10.4.64.2) como servidor e os PCs n18 e n26 como clientes DHCP. Posteriormente, realizaram-se capturas que evidenciam o funcionamento do protocolo, na Figura 11.

```
root@n18:/tmp/pycore.1/n18.conf# ifconfig
eth2: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.4.64.3 netmask 255.255.252.0 broadcast 10.4.95.255
    inet6 fe80::200:ff:feaa:16 prefixlen 64 scopeid 0x20<link>
    ether 00:00:00:aa:00:16 txqueuelen 1000 (Ethernet)
    RX packets 297 bytes 34819 (34.8 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 22 bytes 3196 (3.1 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

Figura 11- protocolo dhcp pc18

- c) Através da Figura 12, podemos verificar a atribuição do endereço IPv4 10.4.64.3 ao PC n18 da Rede B1 atribuído pelo servidor.

Utilizando o Wireshark:

No.	Time	Source	Destination	Protocol	Length	Info
2	5.135702894	0.0.0.0	255.255.255.255	DHCP	342	DHCP Discover - Transaction ID 0x31d4871b
4	6.137207376	10.4.64.2	10.4.64.3	DHCP	342	DHCP Offer - Transaction ID 0x31d4871b
5	6.137438401	0.0.0.0	255.255.255.255	DHCP	342	DHCP Request - Transaction ID 0x31d4871b
6	6.141198065	10.4.64.2	10.4.64.3	DHCP	342	DHCP ACK - Transaction ID 0x31d4871b

Figura 12 - Protocolo DHCP pc n18



Na Figura 12, podemos verificar a atribuição do endereço IPv4 10.4.64.4 ao PC n26 da Rede B1 atribuído pelo servidor.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	0.0.0.0	255.255.255.255	DHCP	342	DHCP Discover - Transaction ID 0xe49ae764
2	0.000724276	00:00:00_aa:00:2b	Broadcast	ARP	42	Who has 10.4.64.1? Tell 10.4.64.2
3	2.760166499	0.0.0.0	255.255.255.255	DHCP	342	DHCP Discover - Transaction ID 0xe49ae764
4	4.095346284	fe80::ec7a:5aff:feb...	ff02::2	ICMPv6	70	Router Solicitation from ee:7a:5a:b7:aa:3b
5	9.479948967	0.0.0.0	255.255.255.255	DHCP	342	DHCP Discover - Transaction ID 0xe49ae764
6	9.480455967	00:00:00_aa:00:2b	Broadcast	ARP	42	Who has 10.4.64.4? Tell 10.4.64.2
7	10.480857620	10.4.64.2	10.4.64.4	DHCP	342	DHCP Offer - Transaction ID 0xe49ae764
8	10.481096080	0.0.0.0	255.255.255.255	DHCP	342	DHCP Request - Transaction ID 0xe49ae764
9	10.484280725	10.4.64.2	10.4.64.4	DHCP	342	DHCP ACK - Transaction ID 0xe49ae764
10	10.494759145	00:00:00_aa:00:2b	Broadcast	ARP	42	Who has 10.4.64.4? Tell 10.4.64.2
11	10.494772430	00:00:00_aa:00:2a	00:00:00_aa:00:2b	ARP	42	10.4.64.4 is at 00:00:00:aa:00:2a
12	10.494782663	10.4.64.2	10.4.64.4	ICMP	62	Echo (ping) request id=0x48a8, seq=0/0, ttl=64 (reply in 13)
13	10.494791992	10.4.64.4	10.4.64.2	ICMP	62	Echo (ping) reply id=0x48a8, seq=0/0, ttl=64 (request in 12)
14	15.871420737	00:00:00_aa:00:2a	00:00:00_aa:00:2b	ARP	42	Who has 10.4.64.2? Tell 10.4.64.4
15	15.871491247	00:00:00_aa:00:2b	00:00:00_aa:00:2a	ARP	42	10.4.64.2 is at 00:00:00:aa:00:2b
16	36.803263213	fe80::200:ff:feaa:16	ff02::2	ICMPv6	70	Router Solicitation from 00:00:00:aa:00:16

Figura 13 - Protocolo DHCP pc n26

O protocolo DHCP é composto por quatro fases:

1. Discover - O cliente envia uma mensagem DHCP Discover em broadcast para procurar servidores DHCP disponíveis na rede.
2. Offer - Um servidor DHCP responde com uma mensagem DHCP Offer, onde propõe um endereço IP.
3. Request - O cliente responde com uma mensagem DHCP Request, indicando que aceita a proposta daquele servidor específico.
4. ACK - Por fim, o servidor envia uma mensagem DHCP ACK a confirmar a atribuição do IP e a finalizar a configuração do cliente.

Nas imagens obtidas no wireshark, observa-se uma sequência completa do protocolo DHCP. Este processo permitiu que os PCs definidos como clientes obtivessem automaticamente um endereço IP válido a partir de um servidor DHCP na rede B1.



4. Uso das camadas de rede e transporte por parte das aplicações

Após ativar um servidor HTTP num dos PCs da rede local C, procedeu-se ao teste a partir de um PC da rede E:

```
root@n22:/tmp/pycore.1/n22.conf# wget 10.4.104.2
--2025-05-24 21:01:37-- http://10.4.104.2/
Connecting to 10.4.104.2:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 225
Saving to: 'index.html'

index.html      100%[=====>]      225  --.-KB/s   in 0s
2025-05-24 21:01:37 (64,1 MB/s) - 'index.html' saved [225/225]
```

Figura 14 - servidor http

No wireshark conseguiu verificar-se que o pc 10.4.104.2 da Rede C estava configurado:

No.	Time	Source	Destination	Protocol	Length	Info
8	5.392336774	10.4.0.2	10.4.104.2	HTTP	172	GET /index.html?nome=Eric&ultimo=Bafa HTTP/1.1
10	5.393754513	10.4.104.2	10.4.0.2	HTTP	494	HTTP/1.1 200 OK

Frame 8: 172 bytes on wire (1376 bits), 172 bytes captured (1376 bits) on interface eth0, id 0
Ethernet II, Src: 00:00:00:aa:00:21 (00:00:00:aa:00:21), Dst: 00:00:00:aa:00:23 (00:00:00:aa:00:23)
Internet Protocol Version 4, Src: 10.4.0.2, Dst: 10.4.104.2
Transmission Control Protocol, Src Port: 57346, Dst Port: 80, Seq: 1, Ack: 1, Len: 106
Hypertext Transfer Protocol
GET /index.html?nome=Eric&ultimo=Bafa HTTP/1.1\r\n
Host: 10.4.104.2\r\n
User-Agent: curl/7.81.0\r\n
Accept: */*\r\n
\r\n
[Full request URI: http://10.4.104.2/index.html?nome=Eric&ultimo=Bafa]
[HTTP request 1/1]
[Response in frame: 10]

Figura 15 - http no pc da rede C

No pc n11 da rede A:

No.	Time	Source	Destination	Protocol	Length	Info
5	53.464918676	10.4.0.2	10.4.104.2	HTTP	172	GET /index.html?nome=Eric&ultimo=Bafa HTTP/1.1
7	53.466176574	10.4.104.2	10.4.0.2	HTTP	494	HTTP/1.1 200 OK

Frame 5: 172 bytes on wire (1376 bits), 172 bytes captured (1376 bits) on interface eth0, id 0
Ethernet II, Src: 00:00:00:aa:00:04 (00:00:00:aa:00:04), Dst: 00:00:00:aa:00:27 (00:00:00:aa:00:27)
Internet Protocol Version 4, Src: 10.4.0.2, Dst: 10.4.104.2
Transmission Control Protocol, Src Port: 47952, Dst Port: 80, Seq: 1, Ack: 1, Len: 106
Hypertext Transfer Protocol
GET /index.html?nome=Eric&ultimo=Bafa HTTP/1.1\r\n
Host: 10.4.104.2\r\n
User-Agent: curl/7.81.0\r\n
Accept: */*\r\n
\r\n
[Full request URI: http://10.4.104.2/index.html?nome=Eric&ultimo=Bafa]
[HTTP request 1/1]
[Response in frame: 7]



b) Foi utilizada a ferramenta Wireshark para capturar o tráfego entre um pc da rede A (10.4.0.2) e o servidor HTTP localizado na rede C (10.4.104.2).

Durante a captura, foi possível observar:

- O estabelecimento da ligação TCP, visível através da troca de pacotes SYN, SYN-ACK e ACK entre cliente e servidor.
- O pedido HTTP, onde o pc 10.4.0.2 solicita a página com GET /...HTTP/1.1
- A resposta HTTP, onde o servidor responde com HTTP/1.1 200 OK.

No pc da rede A:

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	10.4.0.2	10.4.104.2	TCP	74	60440 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM=1 TSval=1058312705 TSecr=0 WS=128
2	0.000127051	10.4.104.2	10.4.0.2	TCP	74	80 → 60440 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0 MSS=1460 SACK_PERM=1 TSval=3624478903 TSecr=10...
3	0.000359931	10.4.0.2	10.4.104.2	TCP	66	60440 → 80 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=1058312706 TSecr=3624478903
4	0.000474137	10.4.0.2	10.4.104.2	HTTP	172	GET /index.html?nome=Eric&ultimo=Bafa HTTP/1.1
5	0.000474137	10.4.104.2	10.4.0.2	TCP	66	80 → 60440 [ACK] Seq=1 Ack=107 Win=65152 Len=0 TSval=3624478903 TSecr=1058312706
6	0.001668972	10.4.104.2	10.4.0.2	HTTP	494	HTTP/1.1 200 OK
7	0.002257036	10.4.0.2	10.4.104.2	TCP	66	60440 → 80 [ACK] Seq=107 Ack=429 Win=64128 Len=0 TSval=1058312707 TSecr=3624478904
8	0.002996443	10.4.0.2	10.4.104.2	TCP	66	60440 → 80 [FIN, ACK] Seq=107 Ack=429 Win=64128 Len=0 TSval=1058312708 TSecr=3624478904
9	0.003363913	10.4.104.2	10.4.0.2	TCP	66	80 → 60440 [FIN, ACK] Seq=429 Ack=108 Win=65152 Len=0 TSval=3624478906 TSecr=1058312708
10	0.003642029	10.4.0.2	10.4.104.2	TCP	66	60440 → 80 [ACK] Seq=108 Ack=430 Win=64128 Len=0 TSval=1058312709 TSecr=3624478906

No pc do servidor http:

No.	Time	Source	Destination	Protocol	Length	Info
2	53.464257183	10.4.0.2	10.4.104.2	TCP	74	47952 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM=1 TSval=1058091104 TSecr=0 WS=128
3	53.464542675	10.4.104.2	10.4.0.2	TCP	74	80 → 47952 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0 MSS=1460 SACK_PERM=1 TSval=3624257301 TSecr=10...
4	53.464634580	10.4.0.2	10.4.104.2	TCP	66	47952 → 80 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=1058091105 TSecr=3624257301
5	53.464918676	10.4.0.2	10.4.104.2	HTTP	172	GET /index.html?nome=Eric&ultimo=Bafa HTTP/1.1
6	53.465252216	10.4.104.2	10.4.0.2	TCP	66	80 → 47952 [ACK] Seq=1 Ack=107 Win=65152 Len=0 TSval=3624257302 TSecr=1058091105
7	53.466176574	10.4.104.2	10.4.0.2	HTTP	494	HTTP/1.1 200 OK
8	53.466296693	10.4.0.2	10.4.104.2	TCP	66	47952 → 80 [ACK] Seq=107 Ack=429 Win=64128 Len=0 TSval=1058091106 TSecr=3624257303
9	53.466816558	10.4.0.2	10.4.104.2	TCP	66	47952 → 80 [FIN, ACK] Seq=107 Ack=429 Win=64128 Len=0 TSval=1058091107 TSecr=3624257303
10	53.467244517	10.4.104.2	10.4.0.2	TCP	66	80 → 47952 [FIN, ACK] Seq=429 Ack=108 Win=65152 Len=0 TSval=3624257304 TSecr=1058091107
11	53.467308767	10.4.0.2	10.4.104.2	TCP	66	47952 → 80 [ACK] Seq=108 Ack=430 Win=64128 Len=0 TSval=1058091107 TSecr=3624257304

d) Procedeu-se à criação do seguinte formulário HTTP:

HTTP Service (Deprecated)

Meta-data

Files Directories Startup/Shutdown Configuration

Config files and scripts that are generated for this service.

File Name

☐ Copy Source File
 ☐ Use text below for file contents

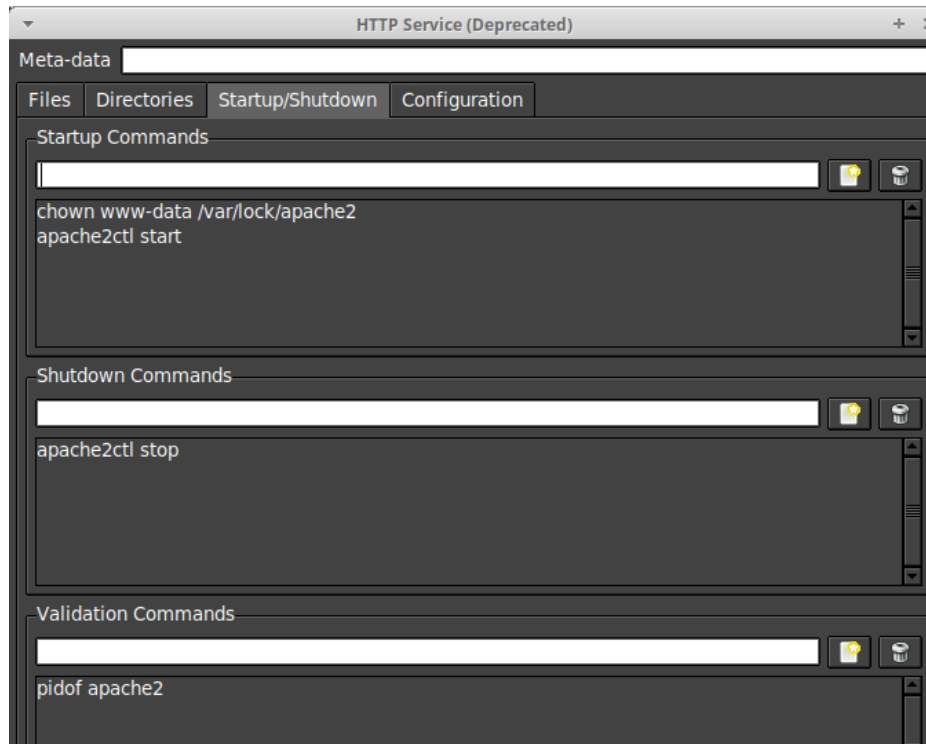
```

<html>
<head>
<title>Formulário</title>
</head>
<body>
<form method="get" action="">
<label>
  nome
  <input type="text" name="nome">
</label>
<label>
  ultimo
  <input type="text" name="ultimo">
</label>
</form>
</body>
</html>

```



Foi ainda necessário configurar a *script* de arranque do serviço HTTP:





5. Interligação via *Network Address Translation*

- a) Foi adicionada uma rede privada à topologia anterior, rede essa que utiliza endereços privados da gama 192.168.1.0/24, como se pode ver na Figura 16.

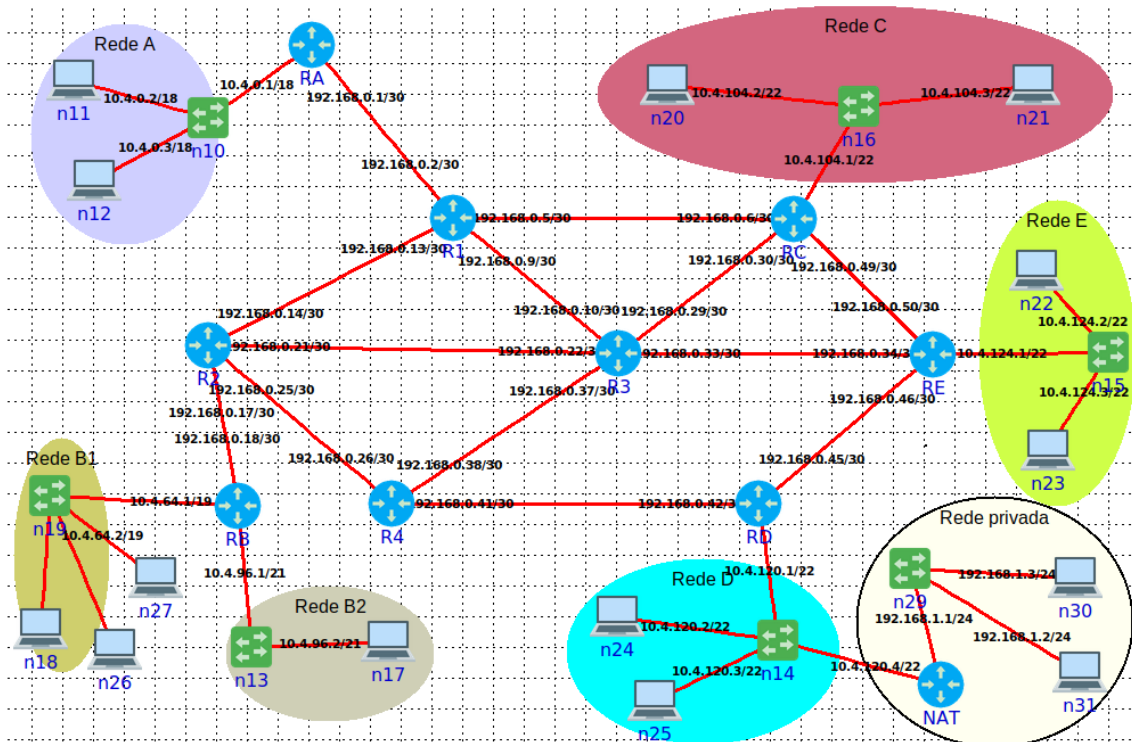


Figura 16 - Topologia com rede privada

- b) De forma a configurar o router NAT, foram realizados os seguintes passos:
- Interface eth0 (ligada à rede externa): 10.4.120.4/22;
 - Interface eth1 (ligada à rede privada): 192.168.1.1/24;
 - Foi definida uma rota default no router NAT para permitir o encaminhamento do tráfego para fora da rede privada

Após realizar estas configurações, testou-se a conectividade (de dentro para fora da rede). Executou-se o comando ping entre os sistemas n30 (IP 192.168.1.3) da rede privada e n20 (IP 10.4.104.2) da rede externa. Analisando a ligação no Wireshark obteve-se os seguintes resultados:



Capturing from eth0 (on NAT)

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

Apply a display filter ... <Ctrl-/>

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	10.4.120.4	10.4.104.2	ICMP	98	Echo (ping) request id=0x1111, seq=1/256, ttl=63 (reply in 2)
2	0.000197133	10.4.104.2	10.4.120.4	ICMP	98	Echo (ping) reply id=0x1111, seq=1/256, ttl=61 (request in 1)
3	1.029853001	10.4.120.4	10.4.104.2	ICMP	98	Echo (ping) request id=0x1111, seq=2/512, ttl=63 (reply in 4)
4	1.029898450	10.4.104.2	10.4.120.4	ICMP	98	Echo (ping) reply id=0x1111, seq=2/512, ttl=61 (request in 3)
5	2.054270758	10.4.120.4	10.4.104.2	ICMP	98	Echo (ping) request id=0x1111, seq=3/768, ttl=63 (reply in 6)
6	2.054316339	10.4.104.2	10.4.120.4	ICMP	98	Echo (ping) reply id=0x1111, seq=3/768, ttl=61 (request in 5)
7	3.078360738	10.4.120.4	10.4.104.2	ICMP	98	Echo (ping) request id=0x1111, seq=4/1024, ttl=63 (reply in 8)
8	3.078409980	10.4.104.2	10.4.120.4	ICMP	98	Echo (ping) reply id=0x1111, seq=4/1024, ttl=61 (request in 7)
9	4.101911514	10.4.120.4	10.4.104.2	ICMP	98	Echo (ping) request id=0x1111, seq=5/1280, ttl=63 (reply in 10)
10	4.101988932	10.4.104.2	10.4.120.4	ICMP	98	Echo (ping) reply id=0x1111, seq=5/1280, ttl=61 (request in 9)
11	5.126064563	10.4.120.4	10.4.104.2	ICMP	98	Echo (ping) request id=0x1111, seq=6/1536, ttl=63 (reply in 12)
12	5.126113503	10.4.104.2	10.4.120.4	ICMP	98	Echo (ping) reply id=0x1111, seq=6/1536, ttl=61 (request in 11)
13	6.150048220	10.4.120.4	10.4.104.2	ICMP	98	Echo (ping) request id=0x1111, seq=7/1792, ttl=63 (reply in 14)
14	6.150094191	10.4.104.2	10.4.120.4	ICMP	98	Echo (ping) reply id=0x1111, seq=7/1792, ttl=61 (request in 13)
15	7.174380268	10.4.120.4	10.4.104.2	ICMP	98	Echo (ping) request id=0x1111, seq=8/2048, ttl=63 (reply in 16)
16	7.174428716	10.4.104.2	10.4.120.4	ICMP	98	Echo (ping) reply id=0x1111, seq=8/2048, ttl=61 (request in 15)
17	8.198549286	10.4.120.4	10.4.104.2	ICMP	98	Echo (ping) request id=0x1111, seq=9/2304, ttl=63 (reply in 18)
18	8.198598332	10.4.104.2	10.4.120.4	ICMP	98	Echo (ping) reply id=0x1111, seq=9/2304, ttl=61 (request in 17)
19	9.221918999	10.4.120.4	10.4.104.2	ICMP	98	Echo (ping) request id=0x1111, seq=10/2560, ttl=63 (reply in 20)
20	9.221967747	10.4.104.2	10.4.120.4	ICMP	98	Echo (ping) reply id=0x1111, seq=10/2560, ttl=61 (request in 19)
21	10.182458326	00:00:00:aa:00:2c	00:00:00:aa:00:15	ARP	42	Who has 10.4.120.1? Tell 10.4.120.4
22	10.182526003	00:00:00:aa:00:15	00:00:00:aa:00:2c	ARP	42	10.4.120.1 is at 00:00:00:aa:00:15
23	10.245906855	10.4.120.4	10.4.104.2	ICMP	98	Echo (ping) request id=0x1111, seq=11/2816, ttl=63 (reply in 24)

▶ Frame 1: 98 bytes on wire (784 bits), 98 bytes captured (784 bits) on interface eth0, id 0
 ▶ Ethernet II, Src: 00:00:00:aa:00:2c (00:00:00:aa:00:2c), Dst: 00:00:00:aa:00:15 (00:00:00:aa:00:15)
 ▶ Internet Protocol Version 4, Src: 10.4.120.4, Dst: 10.4.104.2
 ▶ Internet Control Message Protocol

Através desta captura de pacotes no Wireshark, foi possível observar o funcionamento do NAT. Quando um PC da rede privada iniciou uma comunicação, os pacotes capturados mostraram o endereço de origem como 10.4.120.4 (IP externo do router NAT) e o endereço de destino como 10.4.104.2, pertencente à rede C. Isso confirma que o NAT está a reescrever corretamente os pacotes e a encaminhá-los para a rede externa.



Conclusão

Para a realização deste trabalho prático foi necessário efetuar várias pesquisas e procurar conteúdo que nos ajudasse a concluir os exercícios propostos.

Apesar das dificuldades e desafios enfrentados consideramos que os objetivos foram alcançados.

Em suma, concluímos que o projeto foi essencial para o aprofundamento dos conhecimentos adquiridos na UC.